

GSoC '21 Project Proposal

Monitoring Setup for EvalAI Admins

April 10, 2021

Name and Contact Information

- Name: Aviral Jain
- Location: Alwar, Rajasthan, India
- Timezone: UTC +5:30 Indian Time Zone
- Email: avi.aviral140@gmail.com
- Github: <https://github.com/burnerlee>
- Phone Number: +917290971289
- Education: B.Tech in Electronics and Communication Engineering at Indian Institute of Technology, Roorkee (Batch of 2023)
- [Resume](#)
- [Portfolio](#)
- Skype Name: live:.cid.fc3b6329817ecd68

Statement of Motivation

What is your motivation for participating in GSoC?

I have always been an ardent developer and tried to build something practical with my developing skills. While exploring what the development community has to offer, I came across the concept of Open Source Development, which grabbed my attention to the core of it. Since then, I've been contributing in the small ways I could, like by having all my projects on GitHub from which others could benefit and fixing bugs on others projects. GSoC provides me with an opportunity to work on an open-source project over a well-planned period of around three

months, where I can implement a significant feature with my mentors' support. That's why I am motivated to participate in this program, as it will enhance my knowledge and help me develop new skills.

Why did you choose CloudCV, and why this idea?

I am a member of [SDSLabs](#), a student technical club at IIT Roorkee that constantly tries to innovate and foster software development activities on the campus. Many of my seniors in SDSLabs have been GSoC fellows for CloudCV in recent years and introduced me to the CloudCV community.

Since then, I started to explore projects CloudCV is working on and primarily started with EvalAI. I liked the concept and use case of this project. I read about how this project compares with the other existing alternatives. This motivated me to explore the codebase, understand how things are made to happen in EvalAI, and eventually started to fix issues in the project. My mentors have been supportive throughout the entire phase of my contributions. They've been actively responding to my queries, reviewing my PRs, assigning me issues, etc. They have always supported me in implementing features using my approaches, which allowed me to think freely, and I loved working with this community. This makes CloudCV the obvious choice for me for my GSoC project.

I chose this particular project because it perfectly aligned with my interests and also correlated to what I have worked on in the past. Also, this project provides an excellent opportunity to apply my knowledge and skills on a practical scale. I understand the importance of performance and believe in software being optimised. Setting up a monitoring system in such a large-scale project and visualising its metrics would help us know what changes we need to make to optimise our software's performance. This project aims to cover almost every service running in EvalAI in the monitoring setup. This would give me the hands-on experience to monitor docker containers, back-end APIs, database queries, and the entire system. This project lies in my sphere of interest, and therefore I have a natural inclination towards this project.

What are your expectations from us during and after the successful completion of the program?

I am looking forward to bonding with the community and making some valuable contributions to the project. I would be grateful to be welcomed into the family of CloudCV. I am always open to essential discussions over ideas and their implementations. I would love to receive feedback and suggestions on my contribution to the project, as I believe that the key to quality code is quality feedback and review. For a successful GSoC, I believe that consistent communication is necessary, which would help me blend my mentors' knowledge and experience into my work.

As I mentioned above, the EvalAI community's projects as a whole drive me to make consistent contributions to the work it is doing. The GSoC project allows me to work on the project as a whole, and after its successful completion, I would be keen to take up more important issues and continue my open-source contribution through CloudCV.

What are you hoping to learn?

I expect to build upon my DevOps skillset by setting up a monitoring setup for a large-scale project. I realise that EvalAI depends on a good number of simultaneously running microservices that are interdependent upon each other. Thus, we would need to develop a well-managed and organised monitoring system that would challenge me as a developer and would give me immense satisfaction upon its completion.

This project would also help me get familiarised with some of the more complex components of the program, and eventually, this would add to my development knowledge. Also, staying in continuous touch with such qualified people in the community would be an excellent opportunity for me to learn.

EvalAI is the first open-source large scale project that I'm working on. I'm learning to collaborate on such a large project and what practices we need to follow to maintain a good code. It's codebase is very well managed and has various checks integrated to check for syntax and formatting errors on making a new PR. Many such practices have made it easier for new contributors like me to get started with this project.

Experiences

Have you taken Computer Vision, Machine Learning, Artificial Intelligence, Natural Language Processing, Deep Learning courses?

I have explored Data Science's field to a mild extent by completing some Machine Learning courses and mainly working on projects implementing these algorithms.

- [Stanford - CS 229 Lecture Series](#) by Andrew Ng to get an introduction to ML.
- [Github Issue Labeler Bot Project](#) - A bot that auto assigns labels to issues using the semantics of the issue name
- Teaching an AI to play Flappy Bird using the RL algorithm

What kind of projects have you worked on in the past? What technologies did you use?

- I worked as an intern at [Uniacco](#) as a web developer for two months. They used the Django framework for their backend servers and ReactJs for the frontend. I made many REST APIs for data handling and integrated external services such as AWS, Stripe, Facebook Ads in their main website and CRM. I also set up a monitoring system on their CRM page to view the statistics of their AWS.
- Analysis.ai was a team project built for submission in the Inter IIT Tech Meet 2021. We developed an electron app where the user can generate his/her dataset, apply augmentations, and train models for Traffic Sign Image Recognition. They can then analyse the results with a wide variety of statistics and embedded tensorboard. The backend for the training and predictions was developed on python and then integrated with the app using flask.
- [CricLive](#) was a team project built on Unity as a submission for Inout 7.0. It is a unique application that shows a live cricket match using text commentary and then, using Artificial Intelligence, renders the live game using animations([video](#)). We were also the Runners up out of 131 submissions in [Inout 7.0](#) this year.
- As mentioned above, being a member of [SDSLabs](#), I've actively contributed to many projects like Beast(CTF hosting platform), Coderunner2.0(executes code for competitive

programming platforms in docker environment), some of which have become open-source, for example, [Pinger](#). I am efficient in writing code in GoLang, Python, Javascript, C++, C and their essential frameworks. Also, I've learned how to deploy projects on AWS and containerise them using Docker.

What is your experience with Python and Javascript?

- I got introduced to Javascript during the initial phase of starting development and later started using Python when I started solving some CTF questions. Since then, I have earned a lot of experience in both languages after doing many projects in each of them.
- I've worked on many frontend frameworks on Javascript like ReactJS, VueJs, React-Native, AngularJS, and I can implement designs and UX using any of the frameworks pretty efficiently.
- Thanks to my previous internships, I have gained a good experience in Django and can write servers and APIs efficiently and correctly. It even helped me make some quick PRs and resolve issues while contributing to EvalAI.

CloudCV Contributions

I would like to dedicate my motivation to actively contribute to the projects to the incredible community of CloudCV. The members are up for discussions and reviews over issues and PRs.

Since January, I've been contributing to CloudCV projects [EvalAI](#) and [Evalai-cli](#) and have fixed several issues on the frontend and backend to the best of my abilities.

Development Contributions:

1. Allow users to edit submission metadata after submission: [EvalAI #3239](#) (merged)
2. Add a new field column to display execution time in the leaderboard and make necessary changes in backend APIs and models to fetch the required data: [EvalAI #3317](#) (merged)
3. Integrate feature to allow users to add metadata while submitting the cli: [Evalai-cli #315](#) (merged)
4. Add feature to send mail on challenge creation using zip file for remote evaluation: [EvalAI #3283](#) (approved)

5. Add instructions to make submissions through CLI on frontend_v2: [EvalAI #3367](#) (merged)
6. Fix layout and remove animations from privacy policy page: [EvalAI #3357](#) (merged)
7. Allow toggling show on the leaderboard when the submission status is submitted/running: [EvalAI #3160](#) (merged)
8. Remove Private tag from submission if a private leaderboard is being displayed: [EvalAI #3295](#) (merged)
9. Change layout for selected submission info: [EvalAI #3163](#) (merged)
10. Fix alignment of actions buttons on evaluation page: [EvalAI #3218](#) (merged)
11. Fix alignment of phases list on submit page: [EvalAI #3221](#) (merged)
12. Fix padding of phase selection placeholder in leaderboard page: [EvalAI #3222](#) (merged)
13. Remove submission metadata inconsistency between frontend and cli by fixing the metadata format: [Evalai-cli #319](#) (merged)
14. Fix unhandled error while logging errors due to JWT API: [Evalai-cli #320](#) (merged)
15. Show status of all workers of manage challenge tab: [EvalAI #3308](#) (in progress)
16. Fix cyclic imports in aws files to allow importing models: [EvalAI #3148](#) (merged).
Reopened for a better implementation [EvalAI #3186](#) (in progress)
17. Show private submission toggle on leaderboard even if no submissions are available: [EvalAI #3299](#) (to be reviewed)
18. Fix multiple simultaneous requests to fetch leaderboard data and stop polling on tab change: [EvalAI #3382](#) (to be reviewed)

Documentation & Readme Contributions:

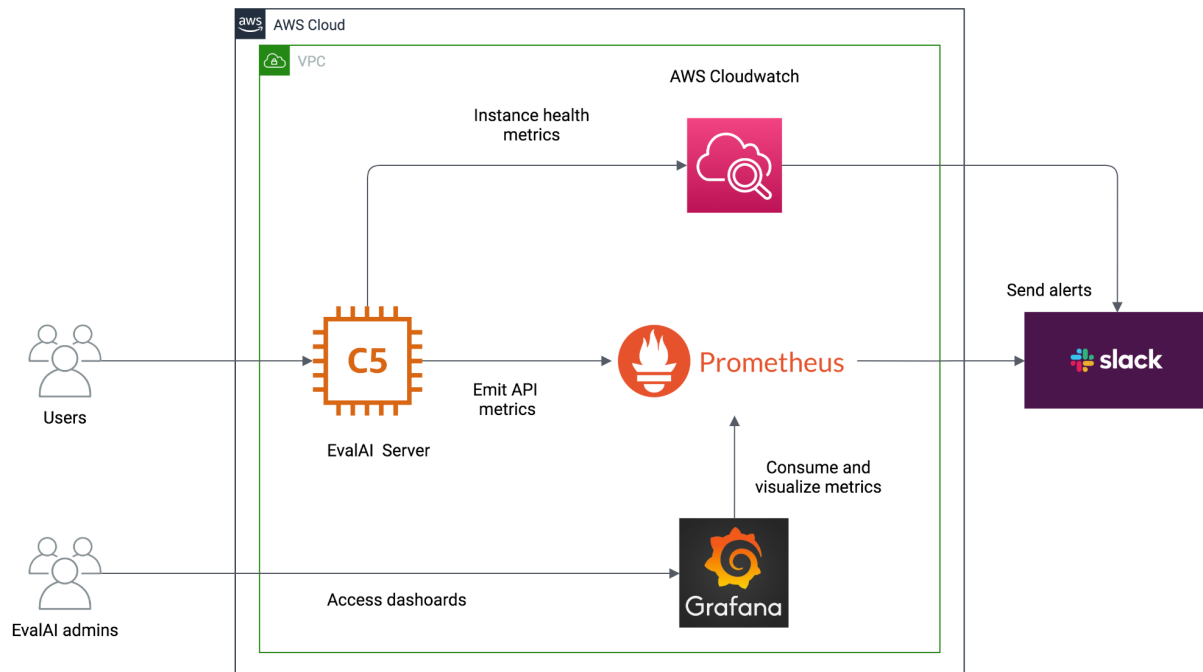
1. Add documentation for leaderboard jobs API: [EvalAI #3140](#) (merged)
2. Add a step to change host while setting up Evalai-cli in the local development environment: [Evalai-cli #306](#) (merged)

Other Open-Source Contributions

1. [Pinger](#): Pinger is an open-source implementation of a status page. I successfully added probes and checks for TCP and UDP protocols. Also, I added some database APIs based on Postgres and GORM.

Project Overview

- As the number of challenges on EvalAI are increasing, we want to improve our services' performance. As a first step, **we will focus on monitoring** and measuring all our services' key metrics. Insights from these will allow us to:
 - **utilise our infrastructure**
 - **improve uptime**
 - **reduce costs efficiently**
- The project will concentrate on setting up **metric reporting** and **alerts infrastructure**, writing REST API's, plotting relevant graphs and **building analytics dashboards** to help EvalAI admins maintain and monitor the services.
- Monitoring the state of our infrastructure and systems is essential for ensuring our services' reliability and stability. Information about our deployments' health and performance helps our team react to issues and gives them the security to make changes with confidence. One of the best ways to gain this insight is with a robust monitoring system that gathers metrics, visualises data, and alerts operators when things appear to be broken. In a large-scale project like EvalAI, being able to tell what is happening within your systems, what resources need attention, and what is causing a slowdown or outage is invaluable.
- Apart from visualising real-time data, this project also aims to integrate a **live alerting system** that would **notify the admins** of the failure or unexpected behaviour of any services of EvalAI. It needs to be flexible to inform admins through multiple mediums and powerful enough to compose thoughtful, actionable notification triggers.
- **Deploying EvalAI on AWS Fargate** would save us from managing our servers which are the EC2 instances currently. With AWS Fargate, there would be no need to scale, provision, or configure virtual machines clusters to run containers. We would cut our costs as Fargate services bill us for CPU or memory requirements for each task in use per second. Whereas in the EC2 instances, we are billed according to the type of EC2 instance chosen on a per hour basis, and some of our EC2 capacity goes wasted as it is not thoroughly utilised.



EvalAI Monitoring System Architecture

Implementation Details

Setup Dockerised Prometheus + Grafana for reporting metrics:

Why Prometheus?

- **Well structured metrics and powerful QL:** The metrics are based on key-value pairs, making them easier to be handled. It provides flexible and accurate time-series data, which can be queried to extract various statistics using the Prometheus Query Language.
- **Service discovery:** The Prometheus server is in charge of periodically scraping the targets so that applications and services don't need to worry about emitting data (metrics are pulled, not pushed). Prometheus also offers exporter clients for services that are not able to expose Prometheus endpoints automatically. For example, for a django based backend in EvalAI, we can use a Prometheus exporter library named `django-prometheus`.

- **Powerful add ons:** Prometheus' community maintains a large number of tools like PromTool, AlertManagers which can be plugged into Prometheus servers very easily and serve their specific tasks.

Setting up dockerised Prometheus would be an easy task considering EvalAI is a containerised application that runs using docker-compose. We would proceed as follows:

Step1: Add Prometheus service and container configurations in the docker-compose.yaml file.

Step2: Define a Prometheus configuration file to scrape metrics from the other services. We can define information such as scrape targets, scrape intervals here.

Prometheus is a powerful tool that can automatically scrape metrics from most of the services or provide tools. In our case, we need to fetch metrics from the evaluation workers and our backend setup. We would need to set up exporters for prometheus on these services to expose a prometheus endpoint to fetch metrics.

Grafana provides out-of-the-box support for Prometheus. Grafana **equips users to query, visualise, and monitor metrics**, no matter where the underlying data is stored. **To install Grafana** in EvalAI, we just need to add this service in our docker-compose file with the necessary configurations. Before making dashboards in our grafana interface, let us add exporters to all the essential services.

Add Django-Prometheus exporter library to EvalAI backend setup:



Prometheus fetching data from Django application using Exporters

We would need to set up an exporter for Prometheus to fetch Django-backend services metrics. [Django-Prometheus](#) is a library that does this job for us. We would proceed as follows:

Step1: We would need to install this library in our backend service using pip and add it to installed apps in Django settings.

Step2: We can now configure it to monitor specific services of the backend. E.g. We can monitor databases, caches or even each model using mixins.

We can **monitor the creation/deletion/update rate** of any model. The best part is that it can be done on an existing model without making migrations. The following graphs can be plotted using metrics received for data such as QPS for each view(for each django app), QPS for each type of request, Request latency, HTTP Methods. Suppose we want to monitor the model Challenge. Its model mixin would look something like this:

```
from django_prometheus.models import ExportModelOperationsMixin
class Challenge(ExportModelOperationsMixin('challenge'), models.Model):
    name = models.CharField()
    category = models.CharField()
```

Add Prometheus counters to report metrics from EvalAI APIs:

- We can add some **custom metrics like a counter** to measure the number of HTTP requests returning a particular status code.
- Django-Prometheus, used as a backend service exporter, allows us to add application-level metrics in our code using the [prometheus_client](#) for python.
- We can define our own metrics function, including a counter which returns the metrics through another endpoint. We can configure it to return metrics of the counter for each response status like 4xx, 2xx, and 5xx separately.

Setup system to fetch metrics from evaluation workers:

1. Monitor the number of submission in the queue

- When a submission is made, each submission message is pushed to an SQS queue, from where a worker service and evaluation starts to pull it. To monitor our SQS, we can follow the procedure mentioned:

- a. AWS SQS emits specific metrics that the CloudWatch service can gather under the namespace AWS/SQS. First, we would **configure our Cloudwatch dashboard** so that all metrics of our Queue are available there.
- b. Now we have to export these SQS metrics available on Cloudwatch to our Prometheus server. We can do this easily by using a Cloudwatch exporter for prometheus. We can **use an exporter like [YACE](#)** to export these metrics from cloudwatch.
- c. The exporter will connect to the AWS Cloudwatch API and pull the metrics. To start with, we would need to set up the configuration file for YACE. These would most importantly contain all the permissions that the exporter needs to access the cloudwatch via API.
- d. YACE is available as a docker image and can be easily added in EvalAI like Prometheus and grafana by adding a service in the docker-compose file.
- e. In the config file, we can specify the queue metrics we need to fetch over a specified period. We can fetch metrics such as **ApproximateNumberOfMessagesVisible**, **ApproximateNumberOfMessagesNotVisible**, **NumberOfMessagesSent**, **NumberOfMessagesReceived**, **NumberOfMessagesDeleted** and many more which are available using cloudwatch.
- f. A sample exporter config file to fetch these metrics from cloudwatch would look something like this:

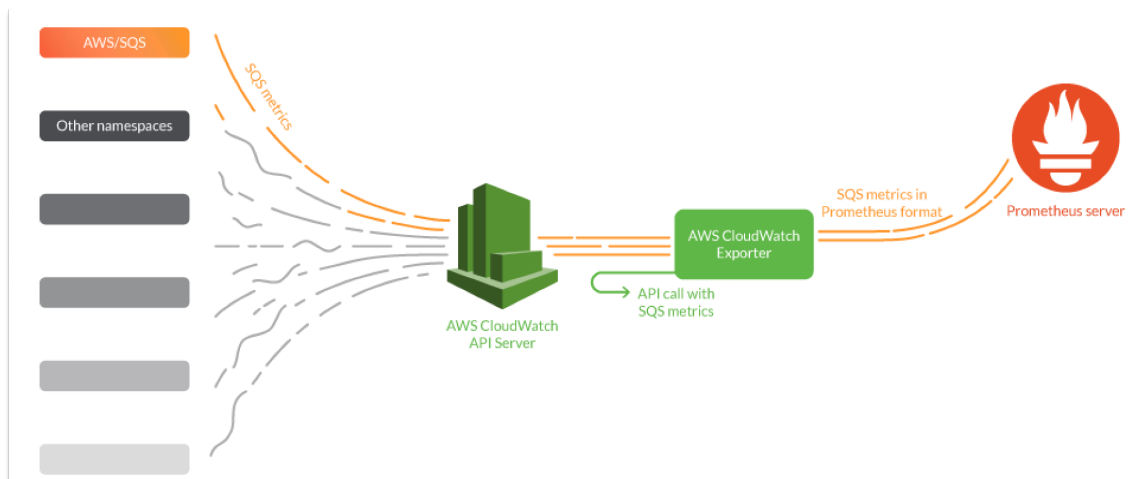
```
discovery:
  jobs:
    - regions:
      - us-east-1
      type: sqs
      enableMetricData: true
      metrics:
        - name: ApproximateNumberOfMessagesNotVisible
          statistics:
            - Average
          period: 300
          length: 3600
        - name: ApproximateNumberOfMessagesVisible
          statistics:
            - Average
          period: 300
```

```

length: 3600
- name: NumberOfMessagesDeleted
  statistics:
    - Sum
    period: 300
    length: 3600
- name: NumberOfMessagesReceived
  statistics:
    - Sum
    period: 300
    length: 3600
- name: NumberOfMessagesSent
  statistics:
    - Sum
    period: 300
    length: 3600

```

The above approach has the following architecture:

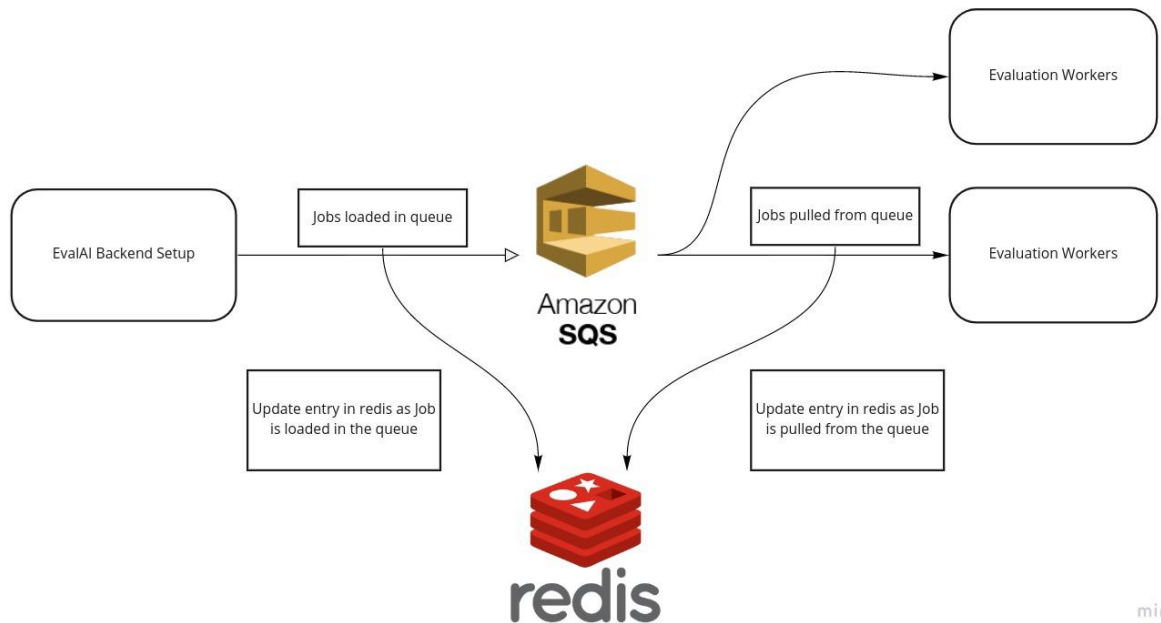


Prometheus fetching SQS metrics from Cloudwatch using the exporter

- We can follow another process rather than the above one to find the number of messages in the queue at a specific amount of time.
 - a. We can add a **Redis container** in EvalAI using docker-compose. In Redis, we can maintain an entry that stores the number of submissions in our queue.
 - b. Whenever a submission is made with correct details, and we publish a message in our queue, we can update this entry in Redis.

- c. When an evaluation worker pulls a job from the queue, we can decrement this entry value.
- d. Using the Grafana plugin Redis-Data-Source, we can display the above data in our grafana dashboards and thus can monitor the number of submissions waiting in the queue.

The architecture is as depicted below:



Schematic diagram to monitor the number of jobs in queue using Redis

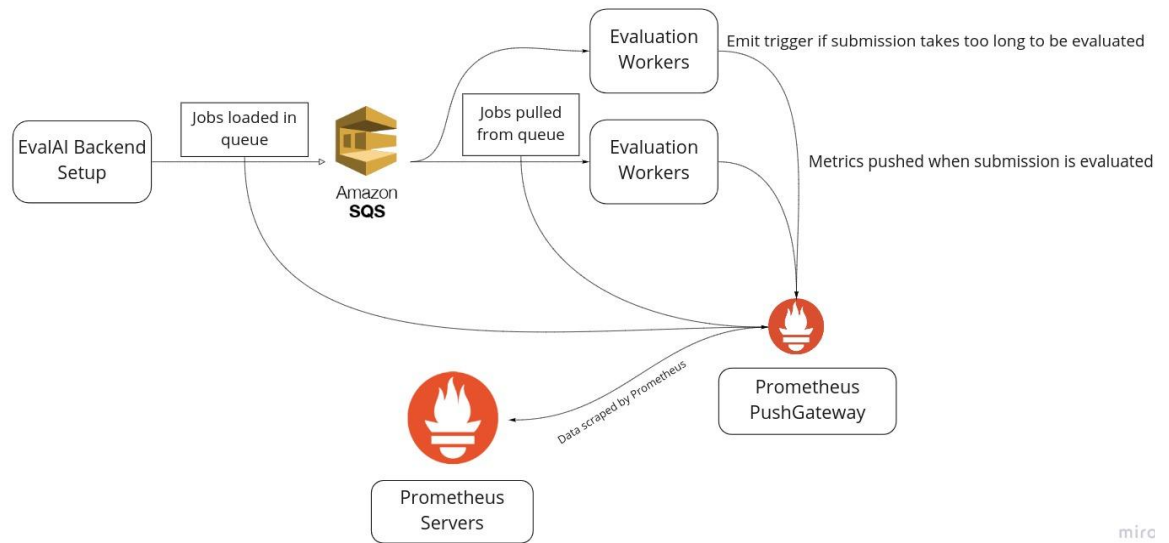
I would recommend going for the first approach because it saves us from manually updating the data for several submissions in the queue. **Cloudwatch provides much more metrics for SQS** to choose from, and we can select which one to monitor.

2. Monitor the number of submissions being processed/failed

- We can monitor the status of jobs processed by the evaluation workers using Prometheus too. These metrics will give us an idea of how long workers are not processing submissions. We can go with the following two approaches for this:
 - a. Using Prometheus client for python, we can define a custom metric endpoint for Prometheus to query the database for the count of total submissions with a

particular status then. A similar logic is used in the backend for checking the number of processing submissions before pushing some job into the queue. We can filter our queries for viewing the submissions with a particular status made in some defined time interval.

- b. **(The preferred approach)** As we use an EC2 instance, we have the power to scale our evaluation workers according to our requirements. Therefore, we can push metrics to Prometheus from each evaluation worker rather than expecting Prometheus to keep track of dynamic worker containers.
- We can use the Prometheus' Push Method, which allows us to push time series data from dynamic workers to an intermediary job known as [Prometheus PushGateway](#), which Prometheus can scrape.
 - [We can use the same](#) Prometheus' Python Client library we used to make custom counters for API metrics to push data to the Prometheus PushGateway.
 - Using the push gateway, we can push metrics for the following events:
 - When an evaluation job is passed into the queue successfully
 - When a worker pulls a job from the queue
 - When a worker finishes evaluating the submission and returns a status
 - We can add a timer and trigger a metric if a submission is being processed for too long
 - We will configure a counter which will increment its value as each worker pushes its status to the Prometheus gateway. This metric can then be scraped away using Prometheus' conventional pull methodology.



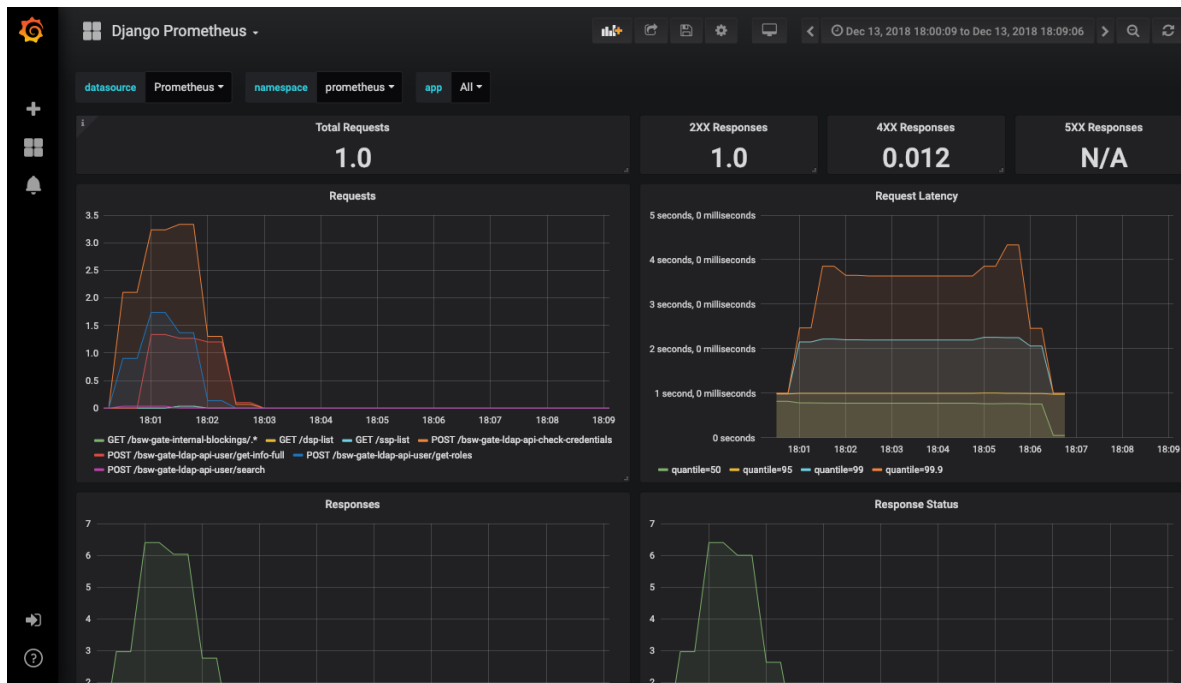
micro

Fetch metrics for submission' status using Prometheus PushGateway

Setting up dashboards in Grafana for visualising metrics:

- Now that we have all our metrics' sources set up, we can configure grafana and display this data using graphs on Grafana dashboards.
- Grafana mostly depends on two main configurations - The Data source, which is the Prometheus server in our case and The dashboards, which visualises our chosen metrics.
- We would segregate our metrics into different dashboards:
 - For each Django app to monitor the status of all its APIs
 - To view metrics for SQS
 - To view metrics for evaluation workers
 - After we integrate the alert manager as discussed ahead, we can display our alerts' logs and display them on a dashboard.
 - Dashboards to view system metrics such as CPU, memory and storage utilisation(if we choose the preferred approach as discussed later)

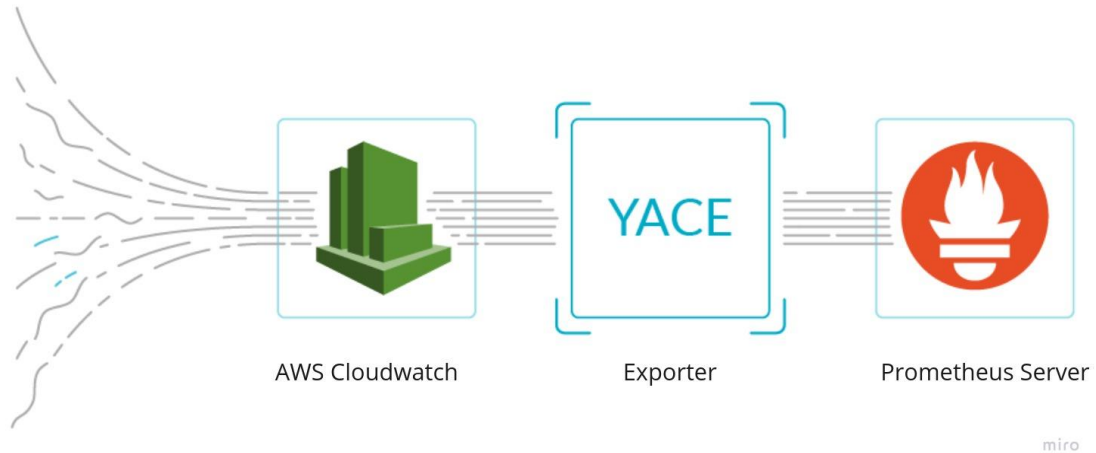
- We can add some additional dashboards if we choose to display metrics fetched using Prometheus plugins like cAdvisor, which fetches the metrics for containers' health and status.
- Grafana is a potent tool and is highly customisable. For every dashboard we create, we can choose all the corresponding sources (here the endpoints from the Prometheus server) and display the received metrics using a wide variety of visualisation tools. For example, a dashboard for Django may look something like this:



A sample grafana dashboard for backend

Setup monitoring for System Metrics:

- We would start by logging into our AWS account and setting up a dashboard on CloudWatch in the AWS/EC2 namespace. We can create any number of dashboards. For example, if we plan to make a dashboard for SQS monitoring, as discussed earlier, we make a separate dashboard for system metrics along with it.
- Once we have our new dashboard, we can choose from all the available metrics we want to display, like CPUUtilization, DiskReadOps, more of which are listed [here](#).



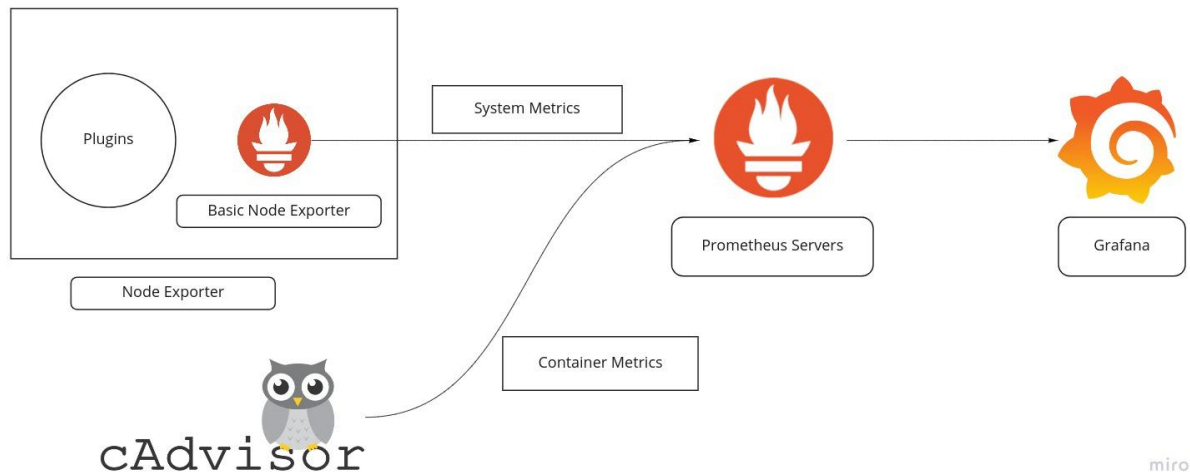
Setup to fetch system metrics from Cloudwatch using Exporter

There is one small problem with the above approach. We use grafana to view metrics of all the other services. We can export these metrics from Cloudwatch to our Prometheus servers and display them on Grafana and the other metrics. This is similar to how we monitor SQS, as discussed previously. We would use the same Exporter for cloudwatch to Prometheus with the AWS/EC2 namespace this time and would be able to get the required data.

But this accessibility comes with a cost. Each API call the exporter makes to Cloudwatch is taken into account and considered while billing. To avoid this, we can use a powerful tool provided by Prometheus' Community named [Node exporter](#).

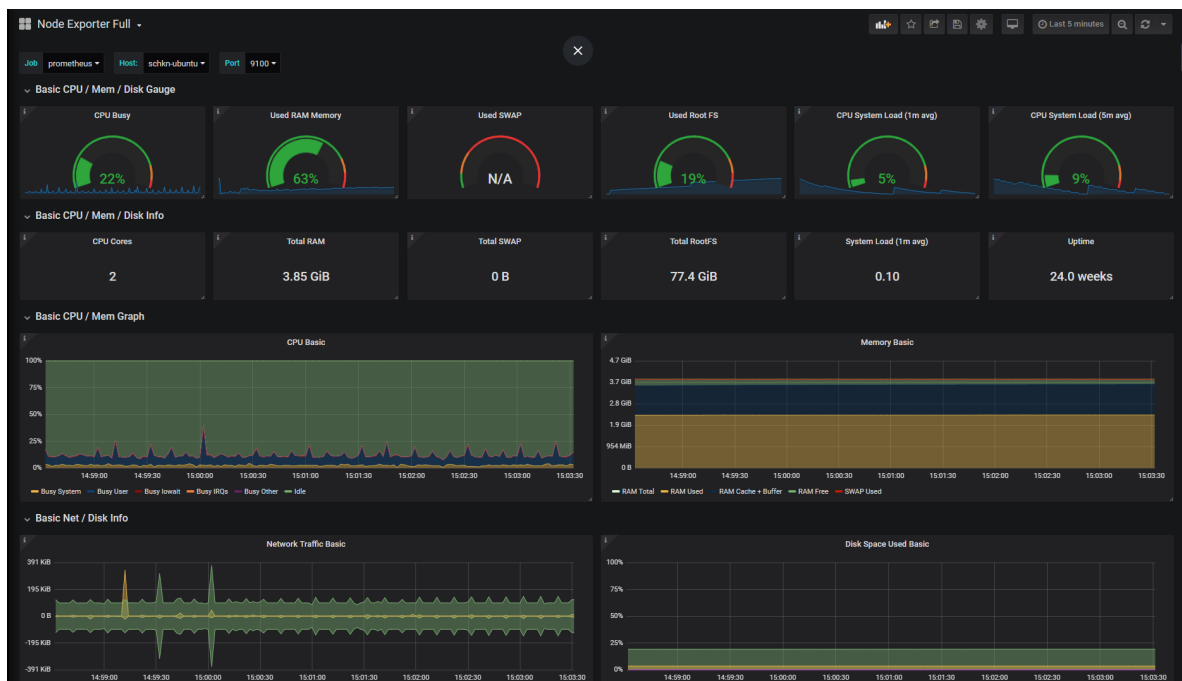
- Node exporter can be installed as a docker image by adding its service in the docker-compose file.
- It adds a metrics endpoint that Prometheus can scrape. We would just need to add the endpoint as a target in its config file.
- We would then monitor almost every possible component of our machine, including CPU, storage and memory utilisation. Some of those few metrics are:
 - CPU / Mem / Disk Gauge / Disc Info / Mem Graphs
 - Basic Net Details
 - System Details
 - Disk Details

- Network Details
 - FileSystem Details
 - You can view more available metrics [here](#)
- We can add more plugins to node-exporter if we would like to monitor our server's more aspects, like analysing our kernel metrics.



Fetch system metrics using Node-Exporter

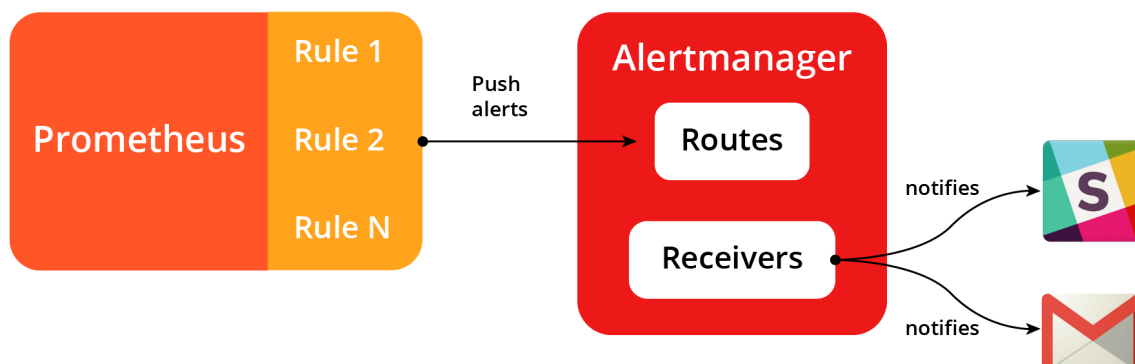
A Grafana dashboard built using this approach would give us a dashboard like:



I would prefer to go with this approach because of the large number of metrics Node-exporter provides at no extra cost. We would view all our metrics at the same places without paying extra for the AWS Cloudwatch API services. While exploring how other large scale open source projects have implemented their monitoring systems, I came across [this project](#). They have set up Node Exporter alongside Grafana and Prometheus to monitor system metrics. Taking inspiration from them and comparing both the options available, I feel that Node Exporter has much more to offer EvalAI when compared to Cloudwatch to help us set up a monitoring system with all metrics in one place.

Setup Real-Time Alerting System:

We can easily plugin an AlertManager in Prometheus to notify admins about some unexpected server response or other critical issues. AlertManager is already installed with Prometheus when we install it using Helm. We can break down the working of the alert manager into two parts:



The architecture of Alerting System using Prometheus

1. We can specify the actual alert conditions(rules in the above diagram) using PromQL(Prometheus Query Language) in the Prometheus component. Prometheus evaluates each condition in a fixed time interval which we can specify.
2. Prometheus evaluates each alert condition repeatedly in a fixed time interval(which we can specify). Whenever any condition evaluates to true, it raises an alert and pushes it to the alert manager. We can instruct the alert manager to notify admins whenever an alert is raised by sending messages through services such as Gmail, Slack, Gitter or any webhook we like. Such notifying endpoints can be added later, though.

We can evaluate conditions for metrics like QPS of backend APIs and the status of evaluation workers. For example, we can add a rule to raise an alert when:

- APIs' exceptions exceed a certain threshold.
- The workers are not evaluating the submissions correctly for a set time.

We can specify details like the alert's severity for each alert condition, the message to be returned, and other information from our metrics. We can divide our alerts into three levels of severity for our monitoring system: **Critical**, **HighPriority** and **ImportantIssue** and can handle each of them differently. For example, we need to notify each admin instantly of a critical issue using the Twilio service to inform them directly on their phone. We don't need to spend on these services for less urgent alerts, and a slack message or mail would suffice.

We can add tags to each alert to classify them. This way, we can notify each admin and some maintainers responsible for any issue related to that tag. For example, for an alert on evaluation workers, we can inform each admin and some maintainers accountable for this issue for an alert on evaluation workers.

An example of Prometheus alert rule would be something like:

```
groups:
- name: backend-down
  rules:
  - alert: No-Backend-QPS
    expr: all_backend{job="qps"} == 0
    for: 5m
    labels:
      severity: critical
      k8s-component: django
    annotations:
      Description: No queries processed by the server in the last 5 minutes.
      Summary: Server is down or not processing requests.
```

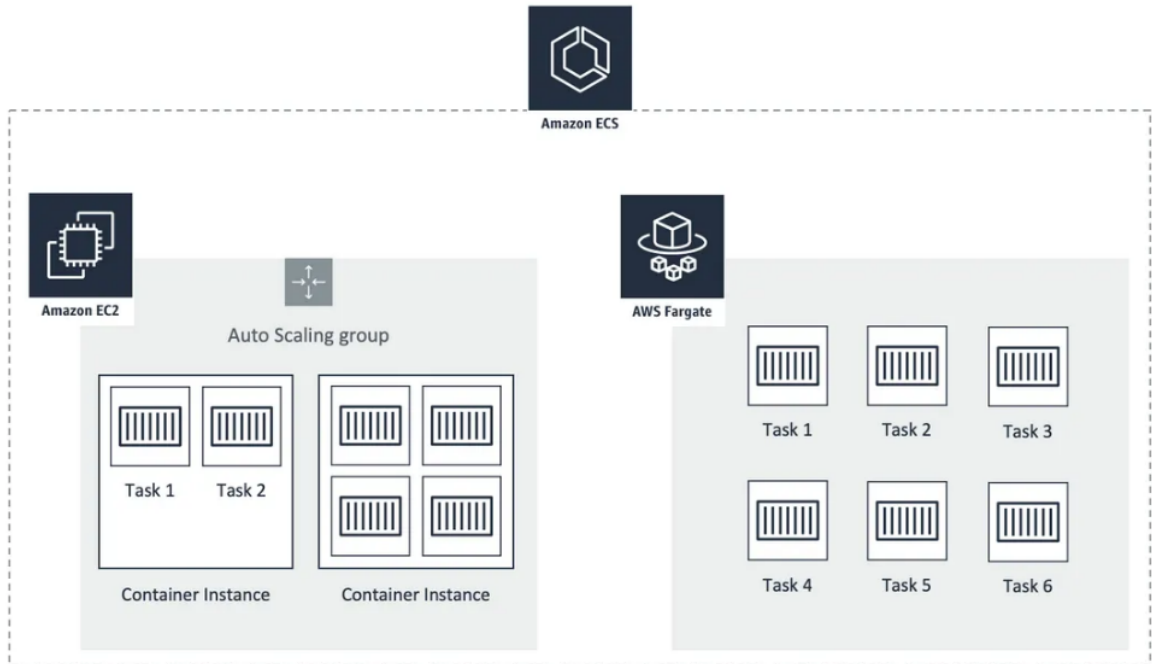
Writing Units Tests for Alert APIs

It's always a good idea to add unit tests to test our code before deployment. EvalAI and EvalAI-CLI, both the projects have unit tests written for all their APIs. I added some units tests along with features implemented through my initial contributions to EvalAI (for example, [EvalAI-CLI #315](#)). We plan to do the same for our monitoring setup

- We would need to add units tests to our backend set up at the following places:
 - Custom metrics' functions defined using Python's Prometheus Client for setting counters.
 - Triggers for pushing metrics to Prometheus on certain events
- For each Prometheus alert file, we can write a test file that would execute the alert file for the set of provided metrics and check if it returns the expected result. [PromTool](#) is a simple yet powerful tool provided by Prometheus for the same. It enables us to check the syntax and working of our alert file with a single command.

Setting up EvalAI deployments on AWS Fargate:

- Currently, EvalAI uses an EC2 instance to deploy its services on AWS. We have to manage our EC2 instance server ourselves by scaling containers manually. For example, we manually scale our evaluation workers using the BOTO3 API when a submission is made. On the other hand, AWS Fargate provides seamless scaling. We don't need to worry about provisioning resources for container apps. We just have to define the application requirements like CPU usage or memory etc., for each task, it scales our requirements in a highly available manner. We no longer have to decide when to scale requirements and pack them for optimal utilisation.
- EC2 bills us based on the type of EC2 instance used on an hourly basis. The problem with this is that even if we are not using 100% of our resources for our instance, we are paying for it, and it goes wasted. It's our responsibility to make sure that our containers are condensed into our instance; otherwise, it is a waste of money. We can view a statistical comparison(done by AWS itself) of the cost involved with both services.



AWS EC2/Fargate Architecture

- EC2 and Fargate both are launch types in the AWS ECS - Elastic Container Service. To migrate our project to the Fargate services, we would need to do the following:
 - We need to change our launch type to Fargate from EC2 while running a task
 - We need to change our network mode from bridge to awsvpc. Each task gets its elastic networking interface and IP address so that multiple applications or copies of a single application can run on the same port number without any conflicts.
 - Other than these significant changes, there are some additional configurations to be changed according to the working of Fargate.
 - Fargate uses the same API actions as ECS, so there should not be many changes in our backend APIs and utils for handling evaluation workers. I'll make the necessary changes in the configurations and scaling rules to make the best use of Fargate's features.

Deliverables:

I plan to deliver the following features to EvalAI during the coding period through the project implementation described above. I am confident that I would integrate all these features and their proper testing in the program duration.

1. Setting up dockerised Prometheus, Grafana to report and visualise metrics
2. Add django-prometheus package to EvalAI backend setup to add prometheus endpoint to scrap metrics
3. Add model mixins using Django-Prometheus to report creation/deletion/update query rate for all the models
4. Setting up Prometheus counters to report metrics from EvalAI APIs to get metrics like the number of HTTP requests with response 4xx, 5xx, 2xx.
5. Add Prometheus counters to report metrics from evaluation workers to get metrics like the number of submissions in queue, the number of submissions being processed, failed, etc. These metrics will give us an idea of how long workers are not processing submissions.
6. Create dashboards and plot graphs for all reported metrics on grafana. Each EvalAI app will have a separate dashboard displaying all metrics for its APIs.
7. Setup Cloudwatch or Node Exporter in Prometheus to view system metrics like CPU, memory and storage utilisation.
8. Setup an alerting monitor and configure alerts for the metrics reported by EvalAI APIs, evaluation worker and system health to notify admins about critical issues like:
 - Workers are not processing submissions.
 - Workers processing submissions but marking every one of them as failed
 - No QPS on backend; i.e. server is down or not processing requests
 - The number of exceptions thrown by APIs exceeds thresholds
9. Write tests for monitoring APIs
10. Setting up EvalAI deployments on AWS Fargate.

Project Timeline

Pre GSoC Period	
April 14th - May 16th	• I would be completing my remaining PRs on EvalAI and EvalAI-CLI (if any) before the community bonding period • Resolving new issues and try to get more familiar with the section of code that deploys evaluation worker • Going through documentation of Prometheus and explore some Grafana dashboards to choose what would be the best for EvalAI
Community Bonding Period	
May 17th - June 6th	• I plan to get to know more about the people in the EvalAI community. I would like to discuss how they plan to develop EvalAI further and what features are they looking forward to • Get access to all necessary resources(if any), for example, a collaborated AWS account where I can build dashboards for setting up Cloudwatch • I would also like to start with my project's first task during this period
Coding Period	
June 7th - June 10th	• Setting up dockerised Prometheus and Grafana on EvalAI • Write the config files for each and make sure Prometheus can scrape data from available endpoints • Visualise these metrics through Grafana to check if it is integrated with Prometheus successfully
June 11th - June 15th	• Integrate Django-prometheus into EvalAI backend setup to export API metrics

	• Add model mixins to report creation/deletion/update query rate for all the models • Add unit tests wherever required
June 16th - June 19th	• Add Prometheus counters to report metrics from EvalAI APIs • Add any unit tests wherever required
June 20th - June 22nd	• Buffer time to complete previous tasks(if any) • Would make some dashboards on Grafana with the metrics available till that point
June 23rd - July 6th	• Add Prometheus counters to report metrics from evaluation workers to get metrics like the number of submissions in queue, the number of submissions being processed, failed, etc. • Add unit tests wherever required
July 6th - July 12th	• Setup dashboards on grafana and display data using graphs would continue this task during the first evaluation phase • Buffer time to complete any previous tasks(if any)
July 12th - July 16th	Evaluation 1 Phase
July 17th - July 24th	• Setup monitoring system to view system metrics using CloudWatch/Node Exporter • Can setup other Prometheus plugins like cAdvisor to monitor other components(containers in this case)
25th July - 7th August	• Setup an Alert Manager on Prometheus to notify admins in case of unexpected behaviour • Setup all alert conditions for all services in their respective alert files and write units tests for each

	• Configure all endpoints where alerts have to be sent
8th August - 15th August	• Setup EvalAI deployments on AWS Fargate • Make sure PRs for all the above tasks are merged • Continue contributing to the project by taking up new issues
16th Aug - 23rd Aug	Final Evaluation Phase

Time commitment

I would be easily able to work more than 20 hours a week. Due to the corona pandemic, the college's summer vacations have shifted ahead to the entire months of June, July and extending to the first week of August. Therefore, there would be no classes for the first two months of the program's coding period. My college would reopen from 4th August. During this time also, I would easily be able to work more than 20 hours. Also, I won't have any exams in August. Other than this, I am not involved in any part-time jobs/internships. I have no plans for outings during this period as well. So, I would be readily available throughout the period except for around 4-5 days in case of emergencies. I generally work from 11 AM to 1 AM (Indian Standard Time), although I find it flexible to adjust my working hours if required.

References

- <https://github.com/Cloud-CV/GSoC-Ideas/issues/35>
- <https://github.com/Cloud-CV/GSoC-Ideas/wiki/GSOC-2021-Proposal-Template>
- <https://prometheus.io/docs/introduction/overview/>
- <https://grafana.com/docs/>
- <https://docs.aws.amazon.com/cloudwatch/index.html>